

Sample Project Documentation

Date	20 July 2026
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine

Project Description

OptiCrop is an advanced Machine Learning-powered smart agricultural recommendation system designed to support modern, data-driven farming practices by providing accurate and intelligent crop prediction solutions. The system analyzes multiple critical environmental and soil-related parameters — including Nitrogen (N), Phosphorous (P), and Potassium (K) soil nutrient levels, temperature, humidity, soil pH, and rainfall patterns — to determine the most suitable crop for cultivation under specific conditions. By combining agricultural science with artificial intelligence, OptiCrop aims to reduce uncertainty in farming decisions and promote efficient, sustainable agricultural production.

The project trains and compares two supervised classifiers — **Logistic Regression** and **Random Forest** — and automatically selects whichever scores higher on held-out test accuracy as the model served in production. Alongside this, **K-Means clustering with the Elbow Method** is used as a complementary, unsupervised analysis to explore natural groupings in the soil/climate data. This combination of supervised classification and unsupervised exploration gives both an accurate point prediction and a broader understanding of how different crops' conditions relate to one another.

Scenarios

Scenario 1: A farmer enters soil readings of N=90, P=42, K=43, along with a temperature of 20.9°C, humidity of 82%, pH of 6.5, and rainfall of 202.9mm on the **Find Your Crop** page. OptiCrop processes these values through the trained model and instantly recommends **rice** as the most suitable crop, given the high humidity and rainfall.

Scenario 2: An agricultural extension officer enters readings for a drier region — lower rainfall and humidity, moderate NPK levels — and OptiCrop recommends a crop suited to semi-arid conditions, helping the officer advise multiple farmers quickly without manual soil analysis.

Scenario 3: A student/researcher reviews the generated `elbow.png` and `pairplot.png` plots (produced by `train_model.py`) to explore how different soil-climate profiles naturally cluster together, gaining insight into regional crop suitability patterns beyond a single point prediction.

Technical Architecture

Description: OptiCrop uses a modular two-part architecture: an offline training pipeline (`train_model.py`) and an online serving application (`app.py`). The training pipeline loads and cleans the dataset, removes outliers, runs exploratory data analysis, compares Logistic Regression against Random Forest, and persists the winning model to `model.pkl`. The Flask serving app loads that saved model once at startup and exposes Home, About, and Find Your Crop pages, with the `/predict` route running inference on user-submitted values.

(Note: OptiCrop does not use a live relational database; the Entity Relationship Diagram in 3. Project Design Phase/Solution_Architecture.md models the conceptual entities — *SoilSample*, *ClimateReading*, *PredictionRequest*, *CropRecommendation* — used to organize the prediction flow.)

Pre-requisites

- **Python Programming Proficiency:** [Python Documentation](#)
- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Machine Learning with scikit-learn:** [scikit-learn Documentation](#)
- **Data Analysis with pandas/numpy:** [pandas Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools Tutorials](#)
- **Version Control with Git:** [Git Documentation](#)
- **Deployment with Unicorn:** [Unicorn Documentation](#)
- **Development Environment Setup:** VS Code / PyCharm

Project Workflow

Milestone 1: Define Problem and Understanding (Epic 1) - Activity 1.1: Specify the business problem — farmers lack a data-driven way to choose the most suitable crop for their soil and climate conditions. - Activity 1.2: Document business requirements — functional (input collection, model inference, result display) and non-functional (speed, usability, reliability). - Activity 1.3: Conduct a literature survey on existing crop recommendation and smart farming systems, comparing algorithms such as Decision Trees, Random Forest, KNN, Logistic Regression, Neural Networks, and clustering methods. - Activity 1.4: Assess the social and business impact — improved yields, reduced resource wastage, and support for sustainable, data-driven farming.

Milestone 2: Data Collection and Analysis (Epic 2) - Activity 2.1: Load the crop recommendation dataset — 2,200

records, 7 features, 22 crop classes. - Activity 2.2: Import required libraries — pandas, numpy, matplotlib, seaborn, scikit-learn. - Activity 2.3: Inspect the dataset's shape, dtypes, and first rows. - Activity 2.4: Perform univariate analysis — boxplots of feature distributions. - Activity 2.5: Perform bivariate analysis — a correlation heatmap across numeric features. - Activity 2.6: Perform multivariate analysis — a pairplot of features colored by crop label, plus rule-based seasonal crop grouping (summer/winter/rainy).

Milestone 3: Data Pre-processing (*Epic 3*) - Activity 3.1: Handle missing and duplicate values in the dataset. - Activity 3.2: Detect and remove outliers from each numeric feature using the IQR method. - Activity 3.3: Scale numerical features (N, P, K, temperature, humidity, pH, rainfall) using `StandardScaler` inside each model's pipeline. - Activity 3.4: Split the dataset into training and testing sets (80/20 stratified split) to preserve the proportion of each crop class.

Milestone 4: Model Building (*Epic 4*) - Activity 4.1: Run K-Means clustering across a range of k values and plot the Elbow curve to explore natural groupings among soil/climate profiles. - Activity 4.2: Train and compare two classifiers — Logistic Regression and Random Forest — each wrapped in a `StandardScaler` + classifier pipeline. - Activity 4.3: Evaluate both models on the held-out test set using accuracy, precision, recall, and F1-score, then print a full classification report for the better model. - Activity 4.4: Save the best-performing model (plus its expected feature order) to `model.pkl`.

Milestone 5: Application Building (*Epic 5*) - Activity 5.1: Build the HTML pages — `home.html` (landing page), `about.html` (objectives & methodology), and `findyourcrop.html` (input form and result display). - Activity 5.2: Build the Python backend (`app.py`) using Flask, loading `model.pkl` once at startup and exposing `/`, `/about`, `/findyourcrop`, and `/predict` routes. - Activity 5.3: Add client-side validation (`script.js`) so the form checks all fields are filled with valid numbers before submission. - Activity 5.4: Run and test the complete application end-to-end locally, and prepare it for deployment via the included `Procfile` (Gunicorn).

Milestone 6: Conclusion

Milestone 1: Define Problem and Understanding

This milestone establishes the foundation of the project by clearly identifying the business problem — farmers struggling to select the most suitable crop based on soil nutrients and climate conditions. The team documented functional and non-functional requirements, surveyed existing literature on ML-based agricultural systems, and assessed the broader social and business impact of a working solution. This groundwork directly informed the choice of features (N, P, K, temperature, humidity, pH, rainfall) and algorithms used in later milestones.

Milestone 2: Data Collection and Analysis

Activity 2.3: Load and Inspect the Dataset

```
def load_data(path: str = DATA_PATH) -> pd.DataFrame:
    df = pd.read_csv(path)
    print("Dataset shape:", df.shape)
    print("\nColumn dtypes:\n", df.dtypes)
    print("\nFirst 5 rows:\n", df.head())
    return df
```

Activities 2.4-2.6: EDA

```
def plot_boxplots(df, columns, out_path):
    sns.boxplot(data=df[columns])
    plt.title("Feature Distributions (Boxplot)")
    plt.savefig(out_path)

def plot_correlation_heatmap(df, columns, out_path):
    corr = df[columns].corr()
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="YlGnBu", square=True)
    plt.savefig(out_path)

def plot_pairplot(df, columns, out_path, sample_crops=None):
    subset = df[df["label"].isin(sample_crops)] if sample_crops else df
    sns.pairplot(subset[columns + ["label"]], hue="label", diag_kind="hist").savefig(out_path)
```

This analysis (saved as `boxplot.png`, `correlation_heatmap.png`, and `pairplot.png` in `static/images/`) revealed how strongly rainfall and humidity correlate with certain crop types (e.g., rice needing high rainfall/humidity), validating the choice of these features for the recommendation model. A simple rule-based `seasonal_crop_analysis()` further grouped crops into summer/winter/rainy categories for exploratory insight.

Milestone 3: Data Pre-processing

```
def clean_data(df: pd.DataFrame) -> pd.DataFrame:
    df = df.drop_duplicates().reset_index(drop=True)
    return df

def remove_outliers_iqr(df: pd.DataFrame, columns) -> pd.DataFrame:
    cleaned = df.copy()
    for col in columns:
```

```
q1, q3 = cleaned[col].quantile(0.25), cleaned[col].quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
cleaned = cleaned[(cleaned[col] >= lower) & (cleaned[col] <= upper)]
return cleaned.reset_index(drop=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Milestone 4: Model Building

```
def elbow_method(X_scaled, out_path, k_range=range(1, 11)):
    inertias = [KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_scaled).inertia_ for k in k_range]
    plt.plot(list(k_range), inertias, marker="o")
    plt.savefig(out_path)
    return inertias

candidates = {
    "Logistic Regression": Pipeline([("scaler", StandardScaler()), ("clf", LogisticRegression(max_iter=1000))]),
    "Random Forest": Pipeline([("scaler", StandardScaler()), ("clf", RandomForestClassifier(n_estimators=200, random_state=42))])
}
results = [evaluate_model(name, model, X_train, X_test, y_train, y_test) for name, model in candidates.items()]
best = max(results, key=lambda r: r["accuracy"])

with open("model.pkl", "wb") as f:
    pickle.dump({"model": best["model"], "features": feature_cols}, f)
```

Milestone 5: Application Building

app.py (Flask backend excerpt):

```
with open("model.pkl", "rb") as f:
    saved = pickle.load(f)
model = saved["model"]
FEATURE_ORDER = saved["features"]

@app.route("/predict", methods=["POST"])
def predict():
    try:
        values = {f: [float(request.form.get(f, ""))] for f in FEATURE_ORDER}
        input_df = pd.DataFrame(values, columns=FEATURE_ORDER)
        prediction = model.predict(input_df)[0]
        return render_template("findyourcrop.html",
                               prediction_text=f"Best crop for the given conditions is {prediction.capitalize()}")
    except (ValueError, TypeError):
        return render_template("findyourcrop.html",
                               prediction_text="Please enter valid numeric values for all fields.")
```

Home Page: Hero section introducing OptiCrop with a call-to-action linking to Find Your Crop. **About Page:** Explains the project’s objectives, methodology (9-step pipeline), and the seven input features used for prediction. **Find Your Crop Page:** A form for the seven required readings (N, P, K, temperature, humidity, pH, rainfall), a **Predict Best Crop** button, and a result box that displays the recommended crop after submission.

Conclusion

OptiCrop is a machine-learning-powered web application that helps farmers, researchers, and agribusinesses make faster, more confident crop selection decisions. By combining K-Means clustering with the Elbow Method for exploratory soil/climate grouping with a supervised classifier (the better of Logistic Regression or Random Forest) for crop prediction, the project demonstrates how classical ML techniques — supported by careful data cleaning, IQR-based outlier removal, and thorough EDA — can meaningfully improve agricultural decision-making. The project, which covered problem definition, data analysis, preprocessing, model comparison, and deployment as a multi-page Flask web application (with Unicorn-ready deployment via the included `Procfile`), highlights how a structured, phase-based approach can turn a real-world agricultural challenge into a working, usable tool. Looking ahead, OptiCrop has room to grow — adding fertilizer/water optimization recommendations, multi-crop (top-3) suggestions with confidence scores, and cloud deployment for wider farmer access are natural next steps for the platform’s evolution.